RAPYUTA ROBOTICS(RSE)

Technical Round 1 — Interview Prep Document

Rapyuta Robotics · RSE Role · Project Discussion Round

This round is a short, deep dive conversation of about 30 minutes about a project you have built. You will share a public GitHub link, walk the interviewer through your project, and give a quick live demo. This guide shows you exactly how to prepare so you can do your best.

What this round is really about

The interviewer is not testing how good your screens look. They want to see two simple things: how well you understand what is happening behind your project, and whether you can clearly explain why you built it the way you did. Keep these two ideas in mind for everything below.

The #1 rule: explain what happens behind the screen

The front-end (the part users see and click) is easy to understand, so do not spend your time there. The real value is explaining the logic working behind it. Here is what that looks like, using a simple login page as an example:

Surface-level — avoid this	Behind the scenes — do this
"This is the login page. You type your email and password, and it logs you in if they are correct."	*"When you click Login, the app sends your details to the server. Your password is compared with the encrypted password saved in the database. If it matches, the server creates a session token so the app remembers you are logged in."*

Do this for every important feature: explain the steps that happen **after** the user clicks, not just what they see on screen.

Always be ready to explain your "why"

For every technology you used, know the reason you chose it.

Be ready for questions like:

- Why this database? What else could you have used?
- Why this framework, library, or programming language?
- What are the advantages and disadvantages of your choice?

Know your project inside out and ready for follow up and cross questions from your answers

Before the call, make sure you can clearly answer:

- What problem does your project solve?
- Which parts did **you** build yourself? (Be honest about this.)
- How do the different parts connect and work together? (the architecture)
- What was the hardest problem you faced, and how did you solve it?
- If you had more time, what would you improve?

Three things that help you stand out

The team really values these. Show them naturally during the conversation:

- **Be proactive** — do not wait to be asked. Offer trade-offs and mention what you would improve.
- **Be upfront** — be honest about what you built and what you do not know yet.
- **Communicate clearly** — explain the simple version first, then go deeper. Keep answers focused and avoid rambling.

Tip: saying *"I am not sure, but here is how I would find out"* is a strong answer. Guessing is not.

How panelist screened your resume

What this panel is actually looking for (the brief version)

The single biggest pattern across every remark: **this panel does not trust the resume — they open the GitHub and check if the code is real.** Almost every positive remark contains some version of *"code backs resume," "all quals backed," "verified."* Almost every negative remark is the opposite: *"projects ABSENT from GitHub," "no C++/Python in code," "tutorial clone," "one-shot commits."*

Candidates got shortlisted when:

- Their **code actually matched their resume** (this is the #1 deciding factor, repeated again and again).
- The **languages that matter were really in the code** — for this role that means **C++ and Python**, not just listed as keywords.
- They showed **sustained activity** (commits spread across months or years), not a single upload.
- Projects were **built step by step** — many small commits, called *"iterated"* in the remarks — instead of one big dump.
- Projects were **substantial and relevant**: multi-file systems work, OpenCV/computer-vision pipelines, a Redis client, a lock-free scheduler, CUDA, full-stack apps.
- They had **strong DSA / competitive programming** (LeetCode Knight, Codeforces ratings, 500–600+ problems), **real internships**, clear **ownership**, and often a **leadership role** (robotics clubs were valued).

Worth noting: low CGPA and off-field degrees (Civil, ECE) did **not** block anyone — *"Lowest CGPA, highest relevance → Best systems fit."* Real skills beat marks here.

Candidates got rejected or flagged when:

- Flagship resume projects had **no code** on GitHub.
- Claimed C++/Python was missing — GitHub was **Java-only or JS-only**.
- Everything was a **last-minute dump** ("all commits same day," "2-day upload").
- Projects were **tutorial clones or copied** ("study-notion is tutorial clone," "copy-pasted PRs").
- The code **contradicted the resume** (claimed Dijkstra/priority-queue, but the repo was plain JS).
- A **library's code was counted as their own** ("only vendored lib bytes"), or the **wrong GitHub account** was linked.

In one line: *they are testing whether you truly built what you claim, in the languages that matter, over real time.*

Previously asked questions

- Walk me through your project.
- Explain your project end-to-end.
- What problem does your project solve?
- What is your role in the project?
- How did you implement this project?
- How does this particular functionality work?
- How did you implement this feature/module?
- Explain the logic behind your implementation.
- What is the architecture of your project?
- Why did you choose this architecture?
- How do different components of your project interact?
- Why did you use this parameter?
- Why did you choose this approach?
- What are the other parameters or alternatives you considered?
- Why did you not use another approach?
- What difficulties did you face while developing the project?
- How did you resolve those challenges?
- What trade-offs did you make during development?
- What other approaches could you think of for this problem?
- If you had to implement this again, what would you do differently?
- How can you scale this project?
- How would the system handle increased users or data?
- What improvements can be made to this project?
- Can you show the source code?
- Explain this piece of code.
- Why did you write the code this way?
- How well do you understand the code and logic of your project?
- Explain your GitHub personal project.
- What technologies have you used in your project?
- How did you implement FastAPI in your project?
- Explain the HTTP requests used in your project.
- How did you implement JWT?
- How did you implement OAuth?
- What status codes did you use and how did you implement them?
- Which ORM did you use in FastAPI?

Quick checklist before your interview

- ☐ Your project is uploaded to a **public** GitHub link (open it in a private/incognito window to confirm anyone can see it).
- ☐ Your demo works and is ready to share on the call.
- ☐ You can explain what happens behind the screen for each main feature.
- ☐ You have a clear reason for every technology you chose.
- ☐ You have practised explaining your project out loud, in simple words.

Take a breath, speak clearly, and show them how well you understand your own work. You've got this.